

Lab 8 Report:

FINAL PROJECT STRATEGIC PLANNING

Lab 8 Instruction:

https://canvas.uw.edu/files/106242897/download?download_frd=1

Team Members: Madison Ringham, Sean Kawano

Dataset for your project: SDSS17 Stellar Classification Dataset

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
```

```
In [2]: from plotly.offline import init_notebook_mode, iplot
init_notebook_mode(connected=True)
```

Understanding the Data

- how big is the data? ✓
- how is it structured? ✓
- are there any obvious patterns? are these patterns expected? ✓
- Are there any unusual features? ✓
- What will we do if the data is corrupted? ✓
- What about if data is mislabeled? --> account for NaNs! ✓
- if the data is simulated, does it have limitations? --> this doesn't apply to our project ✓
- how many features are there per data sample? ✓
- does the dataset have an equal number of datapoints per label? --> it should! check if every entry has valid points for the features we're considering ✓

```
In [3]: # data size
data = pd.read_csv("star_classification.csv")
print(f'The dataset includes {len(data)} observations, including stars, galaxies, and quasars')
```

The dataset includes 100000 observations, including stars, galaxies, and quasars

```
In [24]: data
```

Out [24]:

	obj_ID	alpha	delta	u	g	r	i	z	run_ID	rerun_ID	cam_col	fiel
0	1.237661e+18	135.689107	32.494632	23.87882	22.27530	20.39501	19.16573	18.79371	3606	301	2	
1	1.237665e+18	144.826101	31.274185	24.77759	22.83188	22.58444	21.16812	21.61427	4518	301	5	
2	1.237661e+18	142.188790	35.582444	25.26307	22.66389	20.60976	19.34857	18.94827	3606	301	2	
3	1.237663e+18	338.741038	-0.402828	22.13682	23.77656	21.61162	20.50454	19.25010	4192	301	3	
4	1.237680e+18	345.282593	21.183866	19.43718	17.58028	16.49747	15.97711	15.54461	8102	301	3	
...	...	...	...	...	...	...	...	...	...	...	...	...
99995	1.237679e+18	39.620709	-2.594074	22.16759	22.97586	21.90404	21.30548	20.73569	7778	301	2	
99996	1.237679e+18	29.493819	19.798874	22.69118	22.38628	20.45003	19.75759	19.41526	7917	301	1	
99997	1.237668e+18	224.587407	15.700707	21.16916	19.26997	18.20428	17.69034	17.35221	5314	301	4	
99998	1.237661e+18	212.268621	46.660365	25.35039	21.63757	19.91386	19.07254	18.62482	3650	301	4	
99999	1.237661e+18	196.896053	49.464643	22.62171	21.79745	20.60115	20.00959	19.28075	3650	301	4	

100000 rows x 18 columns

In [4]:

```
# data structure

print(f'Number of features per observation:{len(data.keys())}')
print(f'Features in the data for each observation{data.columns.tolist()}')
```

Number of features per observation:18

Features in the data for each observation['obj_ID', 'alpha', 'delta', 'u', 'g', 'r', 'i', 'z', 'run_ID', 'rerun_ID', 'cam_col', 'field_ID', 'spec_obj_ID', 'class', 'redshift', 'plate', 'MJD', 'fiber_ID']

The dataset we're using, the SDSS17 Stellar Classification Dataset, includes 100,000 observations each with 18 features. Each observation has one value per feature. One of the features, "class" lists the object type (star, galaxy, or quasar). The data is organized in a single .csv file, with each row representing a single observation, and each column representing the features associated with that observation.

In [5]:

```
# data patterns

def k_means_clustering(data, cluster_size):
    kmeans = KMeans(n_clusters=cluster_size, random_state=42).fit(data)
    centroids = kmeans.cluster_centers_
    rep_id = pairwise_distances_argmin(centroids, data)
    clustering = data[rep_id]
    return clustering

feature_list = ["u", "g", "r", "i", "z", "redshift", "alpha", "delta", "MJD"]

# setting the class labels: galaxies = 0, quasars = 1, stars = 2
label_dict = {"GALAXY":0, "QSO":1, "STAR":2}

# not including the scaling at first
#scaler = StandardScaler()
#data[feature_list] = scaler.fit_transform(data[feature_list])
#threshold = 5
#data = data[~ (data[feature_list]**2 > threshold**2).any(axis=1)]

for feature in data[feature_list].columns:
    mean = data[feature].mean()
    std = data[feature].std()
    d = data[abs(data[feature] - mean) <= 5 * std]

#data = data.sample(100000)

X = data[feature_list].to_numpy()
y = data["class"].replace(label_dict).to_numpy()

galaxies = data[data["class"] == "GALAXY"][feature_list]
quasars = data[data["class"] == "QSO"][feature_list]
stars = data[data["class"] == "STAR"][feature_list]

galaxies_np = galaxies.to_numpy()
quasars_np = quasars.to_numpy()
stars_np = stars.to_numpy()

print(f"number of galaxies: {len(galaxies_np)}")
print(f"number of quasar: {len(quasars_np)}")
print(f"number of stars: {len(stars_np)}")
```

```
galaxy_sample = k_means_clustering(galaxies_np, 30)
quasar_sample = k_means_clustering(quasars_np, 30)
stars_sample = k_means_clustering(stars_np, 30)

clusters = [galaxy_sample, quasar_sample, stars_sample]
```

/var/folders/hw/6n5293rd1yxb4t280d29n1t000000gn/T/ipykernel_54367/636647280.py:29: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
y = data["class"].replace(label_dict).to_numpy()
```

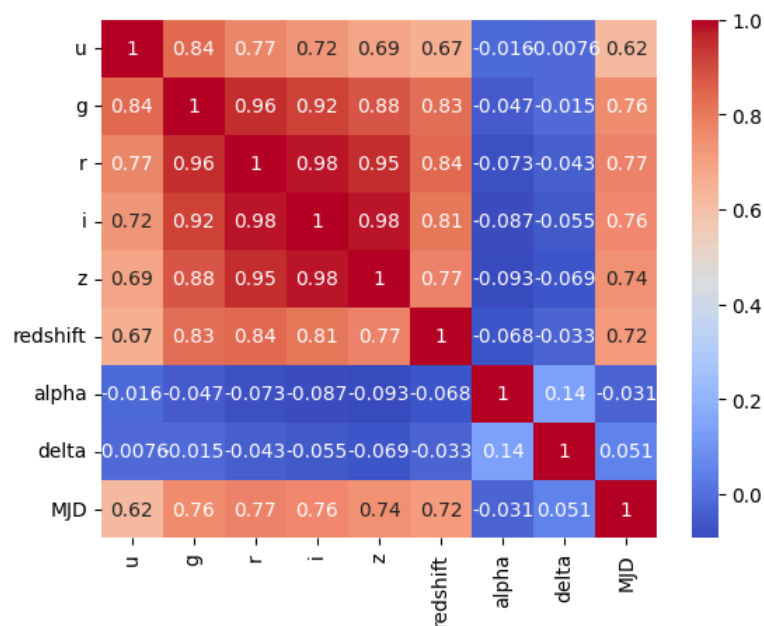
number of galaxies: 59445

number of quasar: 18961

number of stars: 21594

```
In [6]: # making a correlation matrix
sns.heatmap(galaxies.corr(), annot=True, cmap='coolwarm')

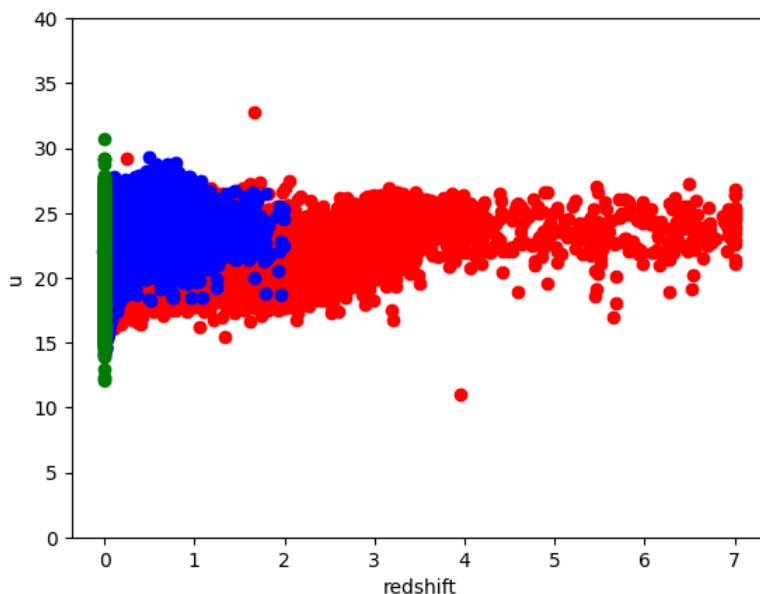
plt.show()
```



Expected Patterns: redshift difference between classes

```
In [34]: plt.scatter(np.array(quasars['redshift']), np.array(quasars['u']), color='r')
plt.scatter(np.array(galaxies['redshift']), np.array(galaxies['u']), color='b')
plt.scatter(np.array(stars['redshift']), np.array(stars['u']), color='g')

plt.ylim(0,40) # there is one outlier in the dataset with ['u'] = -999
plt.xlabel('redshift')
plt.ylabel('u');
```



Looking at the differences in redshift between classes in the dataset, these values are expected. Redshift is an indicator of how far away an object is from Earth, as well as whether an object is moving towards or away from our galaxy and how fast. Objects that are farther away are moving away from us much faster. In general terms, we'll consider redshift as a distance indicator, where the larger the value indicates a greater distance.

We expect stars to have a much lower redshift value (around zero), since stars we observe most likely originate from within the Milky Way galaxy. We also expect some stars to have a slightly negative redshift ('blueshift') indicating that the star is moving towards Earth.

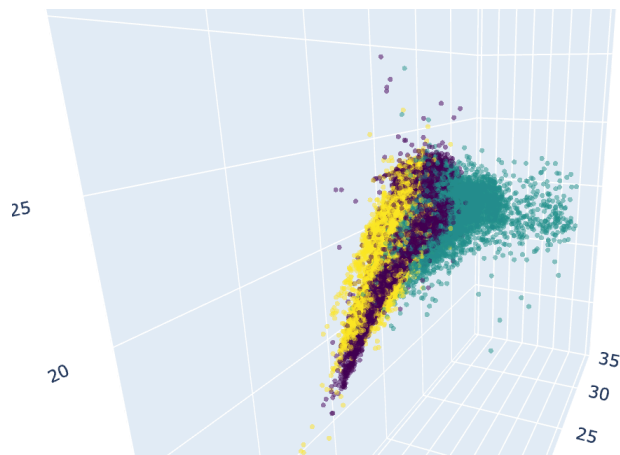
Galaxies and quasars are much farther away, so they would have a greater redshift value. Quasars are much more luminous than galaxies, so we expect that observations in the dataset with large redshifts to be quasars.

The above plot shows the expected redshifts: stars have redshift values around zero, galaxies span redshift 0-2, and quasars span redshifts 0-7.

```
In [59]: labels = y*0.5
         trace = go.Scatter3d(
             x=np.array(data['redshift']),
             y=np.array(data['u']),
             z=np.array(data['z']),
             mode='markers',
             marker=dict(
                 size=2,
                 color=y,
                 colorscale='Viridis',
                 opacity=0.5
             )
         )
         fig=go.Figure(data=[trace])

         fig.update_layout(
             title='3D Scatter Plot',
             scene=dict(
                 yaxis=dict(range=[5,35]),
                 zaxis=dict(range=[5,35]),
                 xaxis_title='Redshift',
                 yaxis_title='u',
                 zaxis_title='z'
             ),
             height=500,
             width=800
         )
         fig.show()
         # fig.show(renderer="browser")
```

3D Scatter Plot



Unusual Features

There doesn't appear to be any unusual features. The coordinates of the object in the sky seem to have a negative correlation with the rest of the data points, but this correlation is so small that the impact of the spatial features will have a negligible impact on the model training. Additionally, it's interesting how the date of the observation has some correlation with the ugriz filter values. Unless we're looking at objects whose brightness varies in time, we'd expect these observations to be time-independent. Additionally, each object has one entry in the database; this prevents the model from capturing time-dependent behavior, so we are deciding not to include the observation time (MJD) in our analysis.

There are filter values with nonsensical values (ex -9999). If we find these values in our data, we need to replace them with NaN.

```
In [57]: # basic script that scans for weird values and replaces them with NaN
data = data.replace(-9999, np.NaN)
data = data.dropna()
```

Corrupted Data

If there is a corrupted file or entry in the database, we want to detect that before feeding the data into our model. To do this, we can scan the database for NaN values or nonsensical fluxes. For example, if an entry in the flux category has a value of -9999, that should be replaced by NaN.

There are 18 features in the entire dataset, but it is possible that some objects don't have valid entries for all of those features. We believe that including all objects in this analysis is important, since it's entirely possible that real-world astronomy data isn't captured in all features. We'd still want to predict the object's class even if an observation doesn't include all of the features present in the dataset, so changing invalid entries to NaN is important so that we can consider all features.

Understanding the Task

- what problem are you trying to solve with your neural network? (classification or regression) ✓
- what is the significance of solving the problem? ✓
- what is the evaluation metric of the neural network model? ✓
- what would constitute as good performance? ✓
- is there a baseline method to compare against after meeting the performance? (classical ML method?) ✓

Goal

We are trying to classify astronomical objects by their class (star, galaxy, or quasar)

This is very important in astronomy, where large-scale sky surveys are generating enormous amounts of data. To complement the influx of data, astronomers need reliable algorithms that identify what an object is based on it's observed characteristics like photometric filter

values. Additionally, classification is important in alert broker systems that alert astronomers about interesting new objects that require follow-up observations. Automating the classification process streamline the follow-up observation process and can help determine what phenomena we observe.

Evaluation metric

We will be evaluating our model on whether it can correctly classify and object in the SDSS17 Stellar Classification dataset. We will look at its overall accuracy, as well as the accuracy in predicting each class.

What is good performance?

Good performance would be a balance between computation time and accuracy. Since our model is more complex than more classic approaches, like decision trees or logistic regression, it takes more time to generate the persistence images we use in our feature extraction. As a result, we aim for our model to out-perform the classical approaches, but also not take forever in the feature extraction or model training portion.

Baseline comparison

We wish to compare our model to a basic logistic regression model, as well as a fully connected network. For the FCN baseline, we plan to cut out the TDA feature extraction and CNN. Based on the accuracy results for our model and the baseline, we can determine the usefulness of our approach.

Baseline models: train a simple MLP on the raw features, see if we get better performance than this.

Developing an initial plan for your project

- what are the inputs and outputs (with dimensions) of the model?
- what data preparation methods do we plan to use?
- what neural network model do you plan to use? (FCN, CNN, RNN), use the raw data in the FCN and the CNN
- what loss function do you plan to use?
- how do you plan to evaluate your model?

The inputs to our neural net model will be three 20x20 images. We pick out the "u," "g", and "redshift," parameters to be the coordinates of the points in our dataset. We first apply normal scaling to each point in our original data and then remove any points that have entries greater than 4. Since we are applying standard scaling, this results in us removing values that were 4 standard deviations away from the mean. Then we use K-means sampling to select a representative sample of each category: Galaxy, Quasar, Star. Then for each point, we append it to the each sample and then compute a persistence image. We plan to use convolutional neural network with the standard cross entropy loss function for classification. Then we will evaluate this model by comparing the accuracy and confusion matrix with other baseline approaches.

In []: